
OPTIMIZACIÓN LOGÍSTICA URBANA

Práctica 4-5: Algoritmia y Complejidad

(Adaptado para la asignatura de Algoritmia y Complejidad, Curso 2025-26)

Temas evaluados:

- Tema 4: Algoritmos de Programación Dinámica
- Tema 5: Recursión Intensiva, Backtracking, Ramificación y Poda

1. Preámbulo y Contextualización

En el entorno actual de logística urbana, empresas como UAH-Route enfrentan desafíos computacionales complejos que requieren soluciones algorítmicas eficientes. Este proyecto estudiantil aborda dos problemas fundamentales interconectados:

1. **Selección Óptima de Pedidos:** Dada una flota limitada de vehículos con capacidad restringida, ¿qué pedidos aceptar para maximizar el beneficio?
2. **Planificación de Rutas Óptimas:** Una vez seleccionados los pedidos, ¿en qué orden entregarlos para minimizar la distancia total recorrida?

Esta práctica se estructura como un proyecto incremental donde los estudiantes deberán desarrollar, analizar y optimizar algoritmos para estos problemas, aplicando los conceptos teóricos de los Temas 4 y 5 de la asignatura.

2. Objetivos de la Práctica

Objetivos Curriculares

- Aplicar Programación Dinámica (Tema 4) al problema de selección de pedidos.
- Implementar Backtracking con Ramificación y Poda (Tema 5) al problema de ruteo.
- Analizar y comparar la complejidad algorítmica de diferentes enfoques.

Objetivos Específicos

1. Modelar el problema logístico mediante estructuras de datos apropiadas.
 2. Diseñar e implementar un algoritmo de **mochila 0/1** usando Programación Dinámica.
 3. Desarrollar un algoritmo de **TSP (Traveling Salesman Problem)** usando Backtracking.
 4. Implementar estrategias de **poda** para reducir el espacio de búsqueda.
 5. Proponer e implementar mejoras algorítmicas basadas en Temas 1-3.
 6. Evaluar experimentalmente el rendimiento con diferentes escenarios.
-

3. Descripción Técnica del Problema

El sistema se compone de **dos módulos integrados**, que deben funcionar de forma coherente:

3.1 Módulo de Selección de Pedidos (Programación Dinámica)

Cada pedido se representa como una estructura:

Pedido = (id, peso, beneficio)

Dado:

- Una lista de pedidos disponibles.
- Una capacidad máxima de carga C.

Se debe seleccionar el subconjunto de pedidos que:

- No exceda C.
- Maximice el beneficio total.

Requisito obligatorio:

La solución debe implementarse mediante **Programación Dinámica**, justificando:

- Estado.
- Relación de recurrencia.
- Casos base.
- Complejidad.

3.2 Módulo de Optimización de Ruta (Backtracking)

Una vez seleccionados los pedidos:

- Cada pedido tiene un punto de entrega.
- La ciudad se modela como un **grafo ponderado** (matriz o lista de adyacencia).
- Se debe encontrar el orden de entrega que minimice la distancia total.

Restricciones adicionales:

- Ventanas de tiempo máximas por pedido (opcional pero valorable).
- Se deben implementar **estrategias de poda** (por distancia acumulada, tiempo excedido, etc.).

Requisito obligatorio:

El algoritmo debe usar **backtracking explícito**, no heurísticas voraces.

4. Actividades a Desarrollar

Fase 1 - Modelado

- Definición de estructuras de datos.
- Representación del grafo urbano.

Fase 2 - Selección (DP)

- Implementación del algoritmo de la mochila.
- Visualización del subconjunto óptimo.

Fase 3 - Ruteo (Backtracking)

- Implementación del algoritmo TSP.
- Inclusión de poda.
- Comparación con versión sin poda (si procede).

Fase 4 - Análisis Algorítmico

- Complejidad teórica.
- Medición empírica (tiempos, nodos explorados).

5. Conjunto de Escenarios de Prueba

Cada grupo debe proponer **al menos 5 escenarios**, documentados y justificados:

1. **Escenario Básico:**
Pocos pedidos ($N \leq 5$), grafo simple.
2. **Escenario de Capacidad Crítica:**
Muchos pedidos pequeños $\rightarrow$ fuerza la DP.
3. **Escenario de Ruteo Complejo:**
Grafo con múltiples caminos similares.
4. **Escenario de Poda:**
Ventanas de tiempo estrictas.
5. **Escenario Libre:**
Caso realista o ampliado propuesto por el grupo.

6. Mejoras Propuestas (Obligatorias mínimo 2)

Tema 1 - Fundamentos y Recursividad:

1. **Generador Recursivo de Escenarios:** Crear instancias de prueba de complejidad creciente usando recursión.
-

2. **Visualización ASCII Animada:** Mostrar evolución de algoritmos paso a paso.

Tema 2 - Algoritmos Voraces:

3. **Comparador Voraz vs DP:** Implementar solución voraz (ordenar por beneficio/peso) y comparar resultados con DP.
4. **Árbol de Recubrimiento Mínimo:** Usar Prim/Kruskal para preprocesar grafo antes de TSP.

Tema 3 - Divide y Vencerás:

5. **Quicksort Personalizado:** Ordenar pedidos usando criterios múltiples (beneficio, urgencia, distancia).
6. **Búsqueda Binaria en Optimización:** Para encontrar capacidad mínima que cubra % de beneficio.

Propuestas Avanzadas:

7. **Floyd-Warshall** para precálculo de todas las distancias mínimas.
8. **Métricas Automáticas** de complejidad empírica (tiempos, nodos explorados, ratio de poda).
9. **Sistema Híbrido DP + Backtracking** con memoización intermedia.
10. **Simulador de Tráfico Dinámico** que afecta distancias en tiempo real.

7. Informe Técnico (15–20 páginas)

Debe incluir:

- Introducción y objetivos.
 - Modelado del problema.
 - Desarrollo de los algoritmos.
 - Análisis de complejidad.
 - Resultados experimentales.
 - Conclusiones.
 - Bibliografía.
-

8. Entregables

Estructura del .zip:

apellido1Apellido2Apellido3_Nombre_PLX.zip/

```
├─ src/
│   ├── main.py           # Script principal
│   ├── dp_seleccion.py   # Módulo Programación Dinámica
│   ├── backtracking_ruta.py # Módulo Backtracking con Poda
│   ├── mejoras/         # Implementaciones de mejoras
│   │   ├── comparador_voraz.py
│   │   ├── floyd_warshall.py
│   │   └── metricas_avanzadas.py
│   └── utils/
│       ├── generador_escenarios.py
│       └── visualizador.py
├─ data/
│   ├── escenarios/       # 5 escenarios en JSON
│   ├── resultados/       # Resultados de ejecución
│   └── tests/
│       ├── test_dp.py
│       └── test_backtracking.py
├─ minutos/              # Minutas bisemanales
│   ├── minuta1.pdf
│   ├── minuta2.pdf
│   └── minuta3.pdf
└─ Informe_Proyecto.pdf  # Informe final
```

Requisitos Técnicos:

- **Lenguaje:** Python
 - **Entrega:** Software totalmente funcional + informe
 - **Documentación:** Comentarios en código, README.md
 - **Pruebas:** Casos de prueba para cada módulo
-